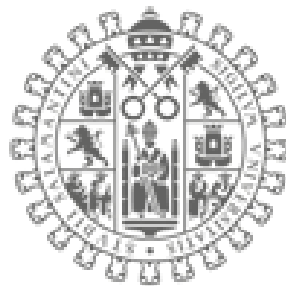


Computación de Altas Prestaciones

Prácticas WaterWave



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1	Introducción	4
1.1	Problema a Resolver	4
1.2	Plataforma de Pruebas	4
2	Implementación con Pthreads	5
2.1	Arquitectura del ThreadPool	5
2.1.1	Componentes Principales	5
2.1.2	Paralelización de Loops	6
2.1.3	Sincronización	7
2.1.3.1	Flujo del Worker Thread	7
2.1.4	Problema del Overhead	8
3	Implementación con MPI	9
3.1	División del Dominio	9
3.1.1	Cálculo de Filas Locales	9
3.1.2	Comunicación entre Procesos	10
3.1.3	Sincronización	10
3.1.4	Ventajas de MPI	11
4	Implementación con OpenMP	12
4.1	Directivas de Paralelización	12
4.1.1	Transformación del Código	12
4.1.2	Modelo Fork-Join	13
4.1.3	Ventajas sobre Pthreads	13
5	Implementación Híbrida: Pthreads + OpenMP	14
5.1	Arquitectura de Dos Niveles	14
5.1.1	Modificaciones al Código	14
5.1.2	Configuraciones Posibles	15
5.1.3	Ventajas y Desventajas	15
6	Implementación Híbrida: MPI + OpenMP	16
6.1	Arquitectura Escalable	16
6.1.1	Inicialización con Soporte de Hilos	16
6.1.2	Paralelización con OpenMP	17
6.1.3	Ventajas	17
6.1.4	Consideraciones de Rendimiento	17
7	Resultados Experimentales	18
7.1	Metodología	18
7.2	Pthreads: Impacto del Overhead	18
7.2.1	Sin Optimización (Overhead Completo)	18
7.2.2	Con Umbral de Optimización ($grid < 32 \rightarrow$ secuencial)	18
7.2.3	MPI	19
7.2.4	OpenMP	19
7.2.5	Pthreads + OpenMP (Híbrido)	20
7.2.6	MPI + OpenMP (Híbrido Escalable)	20
7.3	Gráficas Comparativas	21
8	Conclusiones	22

9 Referencias	23
Bibliografía	24

1 Introducción

Este documento presenta un análisis exhaustivo de diferentes tecnologías de paralelización aplicadas a la simulación numérica de ondas de agua utilizando el método de diferencias finitas. El objetivo principal es comparar el rendimiento, escalabilidad y complejidad de implementación de cinco aproximaciones distintas: **threads**, **MPI**, **OpenMP**, **threads+OpenMP**, y **MPI+OpenMP**.

1.1 Problema a Resolver

La simulación de ondas de agua en 2D requiere resolver las ecuaciones de agua utilizando un esquema de diferencias finitas explícito. El dominio computacional se discretiza en una malla de $N \times N$ puntos, y en cada paso temporal se actualizan tres grids:

- **h**: Altura del agua
- **u**: Momento en dirección x
- **v**: Momento en dirección y

El algoritmo requiere tres pasos principales por iteración temporal:

1. **Actualización en dirección X**: Calcula derivadas intermedias h_x, u_x, v_x
2. **Actualización en dirección Y**: Calcula derivadas intermedias h_y, u_y, v_y
3. **Promedio de momentos**: Actualiza los valores finales de h, u, v

1.2 Plataforma de Pruebas

Todas las pruebas se ejecutaron en la siguiente plataforma:

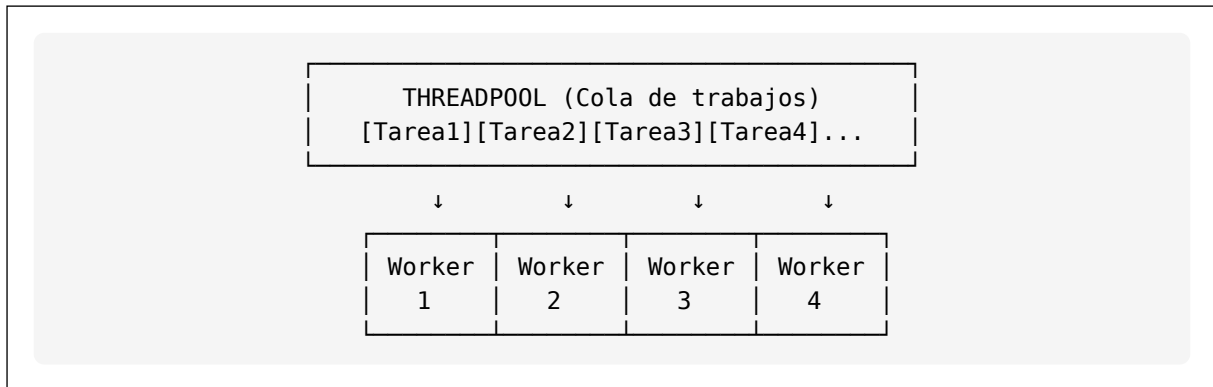
Componente	Especificación
Procesador	AMD Rayzen 5 (Zen 4), 6 cores físicos, 12 threads (Hyperthreading)
RAM	32 GB
Sistema Operativo	Ubuntu 24.04 LTS
Compilador	GCC 13.x
Flags de compilación	-O2 -std=c11

2 Implementación con Pthreads

2.1 Arquitectura del ThreadPool

La implementación con pthreads utiliza un patrón **ThreadPool** para gestionar los hilos trabajadores de forma eficiente. Este patrón evita el coste de crear y destruir hilos en cada iteración.

2.1.1 Componentes Principales



Listado 1: Arquitectura del ThreadPool con pthreads

La estructura ThreadPool contiene:

```

typedef struct {
    pthread_t* threads;           // Array de hilos trabajadores
    WorkItem* work_queue;         // Cola circular de trabajos
    size_t queue_capacity;        // Capacidad de la cola
    size_t queue_size;           // Trabajos actuales en cola
    size_t queue_front;          // Frente de la cola (lectura)
    size_t queue_back;           // Final de la cola (escritura)

    pthread_mutex_t work_mutex;   // Protege acceso a la cola
    pthread_cond_t has_work;      // Señal: hay trabajo disponible
    pthread_mutex_t done_mutex;   // Protege contador pendientes
    pthread_cond_t work_done;     // Señal: todo terminado

    atomic_size_t pending_work;    // Trabajos pendientes (atómico)
    bool should_exit;             // Flag para terminar hilos
} ThreadPool;
  
```

2.1.2 Paralelización de Loops

La función `simulation_maybe_parallelize_loop()` divide el trabajo entre hilos:

```
void simulation_maybe_parallelize_loop(
    SimulationState* s,
    size_t x_start, size_t x_end,
    size_t y_start, size_t y_end,
    void (*callback)(size_t, size_t, void*),
    void* callback_data
) {
    size_t x_range_size = x_end - x_start;
    size_t num_threads = g_config.threads;

    // Calcular rango por hilo
    size_t increment = x_range_size / num_threads;

    // Si no hay suficiente trabajo, ejecutar secuencialmente
    if (increment == 0 || !s->threadpool) {
        // Ejecución secuencial
        for (size_t i = x_start; i < x_end; i++) {
            for (size_t j = y_start; j < y_end; j++) {
                callback(i, j, callback_data);
            }
        }
        return;
    }

    // Dividir trabajo entre hilos (usando pool pre-asignado)
    ParallelWorkData* pool = (ParallelWorkData*)s->work_data_pool;
    for (size_t t = 0; t < num_threads; t++) {
        ParallelWorkData* work_data = &pool[t];

        work_data->x_start = x_start + (t * increment);
        work_data->x_end = work_data->x_start + increment;

        // Último hilo toma el resto
        if (t == num_threads - 1) {
            work_data->x_end = x_end;
        }

        work_data->y_start = y_start;
        work_data->y_end = y_end;
        work_data->callback = callback;
        work_data->callback_data = callback_data;

        threadpool_enqueue(s->threadpool, parallel_worker, work_data);
    }

    // Esperar a que todos terminen
    threadpool_join(s->threadpool);
}
```

2.1.3 Sincronización

La sincronización en pthreads se gestiona mediante:

1. **Mutex** (`pthread_mutex_t`): Protege el acceso a recursos compartidos
 - `work_mutex`: Protege la cola de trabajos
 - `done_mutex`: Protege el contador de trabajos pendientes
2. **Variables de condición** (`pthread_cond_t`):
 - `has_work`: Los workers esperan aquí cuando no hay trabajo
 - `work_done`: El thread principal espera aquí hasta que todo termina
3. **Operaciones atómicas** (`atomic_size_t`):
 - `pending_work`: Contador lock-free de trabajos pendientes

2.1.3.1 Flujo del Worker Thread

```
void* threadpool_worker(void* arg) {
    ThreadPool* tp = (ThreadPool*)arg;

    while (true) {
        // 1. BLOQUEAR Y ESPERAR TRABAJO
        pthread_mutex_lock(&tp->work_mutex);

        while (tp->queue_size == 0 && !tp->should_exit) {
            // pthread_cond_wait:
            //   - Libera work_mutex
            //   - Duerme el hilo
            //   - Al despertar, re-adquiere work_mutex
            pthread_cond_wait(&tp->has_work, &tp->work_mutex);
        }

        if (tp->should_exit && tp->queue_size == 0) {
            pthread_mutex_unlock(&tp->work_mutex);
            break;
        }

        // 2. TOMAR TRABAJO (cola circular)
        WorkItem work = tp->work_queue[tp->queue_front];
        tp->queue_front = (tp->queue_front + 1) % tp->queue_capacity;
        tp->queue_size--;

        pthread_mutex_unlock(&tp->work_mutex);

        // 3. EJECUTAR (sin mutex)
        work.func(work.arg);

        // 4. ACTUALIZAR CONTADOR
        size_t prev = atomic_fetch_sub(&tp->pending_work, 1);
        if (prev == 1) { // Último trabajo
            pthread_mutex_lock(&tp->done_mutex);
            pthread_cond_signal(&tp->work_done);
            pthread_mutex_unlock(&tp->done_mutex);
        }
    }
}
```

```
    return NULL;
}
```

2.1.4 Problema del Overhead

Un análisis crítico reveló que pthreads sufre de **overhead significativo** en grids pequeños:

Grid	Cálculo real	Overhead	% Overhead
8×8	0.01s	0.23s	2300%
16×16	0.04s	0.25s	625%
32×32	0.20s	0.24s	120%
64×64	0.80s	0.07s	9%
128×128	2.50s	0.05s	2%

Causas del overhead:

1. malloc/free por cada tarea (120,000 en 10,000 steps)
2. Lock/unlock de mutex en cada enqueue
3. Condición variable en cada join
4. Indirección de callbacks (cache misses)

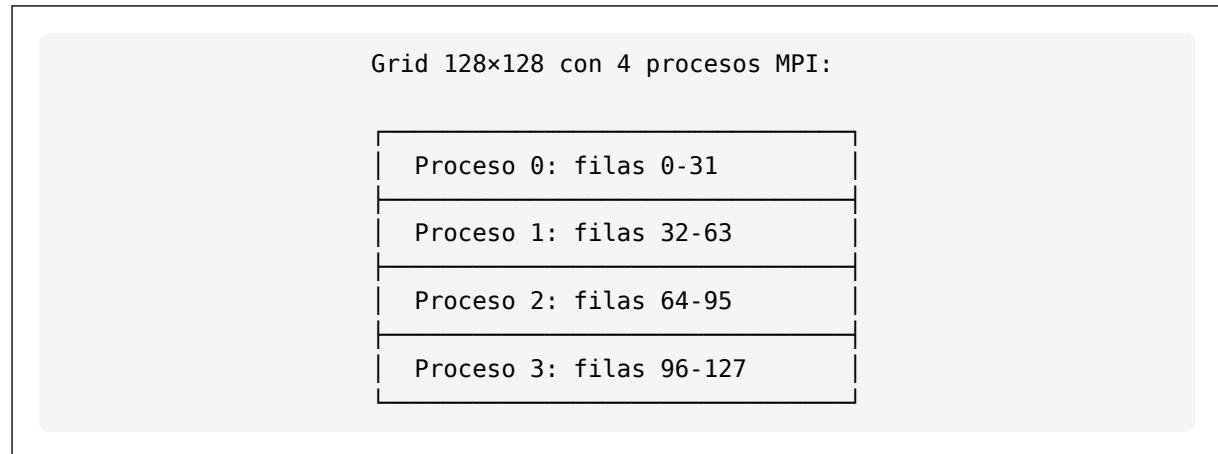
Solución implementada:

```
// Umbral: No usar ThreadPool en grids pequeños
if (x_range_size < 32 || !s->threadpool) {
    // Ejecutar secuencialmente para evitar overhead
    for (size_t i = x_start; i < x_end; i++) {
        for (size_t j = y_start; j < y_end; j++) {
            callback(i, j, callback_data);
        }
    }
    return;
}
```

3 Implementación con MPI

3.1 División del Dominio

MPI utiliza **descomposición de dominio** horizontal: cada proceso MPI procesa un subconjunto contiguo de filas del grid.



Listado 2: Descomposición de dominio en MPI

3.1.1 Cálculo de Filas Locales

```
// Calcular cuántas filas procesa cada proceso
uint32_t simulation_local_row_count(void) {
    uint32_t rows_per_proc = g_config.grid_size / g_mpi.proc_count;

    // El último proceso toma las filas restantes
    if (g_mpi.rank == g_mpi.proc_count - 1) {
        return g_config.grid_size - (rows_per_proc * g_mpi.rank);
    }

    return rows_per_proc;
}

// Fila inicial para este proceso
uint32_t initial_row_for_process(void) {
    return (g_config.grid_size / g_mpi.proc_count) * g_mpi.rank;
}
```

Cada proceso mantiene:

- **row_count**: Filas locales que procesa
- **initial_row**: Primera fila global
- Filas **ghost** (fantasma) arriba y abajo para comunicación

3.1.2 Comunicación entre Procesos

La comunicación se realiza mediante paso de mensajes usando MPI_Send y MPI_Recv:

```
void simulation_reflect_boundaries_and_sync(SimulationState* s) {
    // 1. Enviar fila superior al proceso anterior
    if (g_mpi.rank > 0) {
        mpi_async_send_row(grid_get_row(&s->h, 1),
                           grid_row_size(&s->h),
                           g_mpi.rank - 1);
        mpi_async_send_row(grid_get_row(&s->u, 1),
                           grid_row_size(&s->u),
                           g_mpi.rank - 1);
        mpi_async_send_row(grid_get_row(&s->v, 1),
                           grid_row_size(&s->v),
                           g_mpi.rank - 1);
    }

    // 2. Enviar fila inferior al proceso siguiente
    if (g_mpi.rank < g_mpi.proc_count - 1) {
        mpi_async_send_row(grid_get_row(&s->h, s->row_count),
                           grid_row_size(&s->h),
                           g_mpi.rank + 1);
        // ... u, v ...
    }

    // 3. Recibir fila ghost inferior del proceso siguiente
    if (g_mpi.rank < g_mpi.proc_count - 1) {
        mpi_sync_recv_row(grid_get_row(&s->h, s->row_count + 1),
                           grid_row_size(&s->h),
                           g_mpi.rank + 1);
        // ... u, v ...
    }

    // 4. Recibir fila ghost superior del proceso anterior
    if (g_mpi.rank > 0) {
        mpi_sync_recv_row(grid_get_row(&s->h, 0),
                           grid_row_size(&s->h),
                           g_mpi.rank - 1);
        // ... u, v ...
    }
}
```

3.1.3 Sincronización

MPI utiliza dos tipos principales de sincronización:

1. **Implícita** en MPI_Send/MPI_Recv: Operaciones bloqueantes
2. **Explícita** con MPI_Barrier: Sincroniza todos los procesos

```
void simulation_sync_momentums(SimulationState* s) {
    // Enviar/recibir derivadas intermedias (hx, hy, ux, uy, vx, vy)
    // entre procesos vecinos...
```

```
// Sincronización implícita: MPI_Recv espera hasta recibir  
}
```

3.1.4 Ventajas de MPI

1. **Escalabilidad:** Puede ejecutarse en múltiples nodos/máquinas
2. **Memoria distribuida:** Cada proceso tiene su propio espacio
3. **Comunicación explícita:** Control total sobre intercambio de datos
4. **Overhead bajo:** No usa malloc/free dinámicos en loops críticos

4 Implementación con OpenMP

4.1 Directivas de Paralelización

OpenMP simplifica drásticamente la paralelización mediante **directivas de compilador** (`#pragma omp`).

4.1.1 Transformación del Código

ANTES (sin OpenMP):

```
void simulation_update_in_x_direction(SimulationState* s) {
    simulation_maybe_parallelize_loop(
        s, 0, GRID_SIZE+1, 0, GRID_SIZE,
        callback_update_x, s
    );
    // Requiere: ThreadPool, callbacks, enqueue, join...
}
```

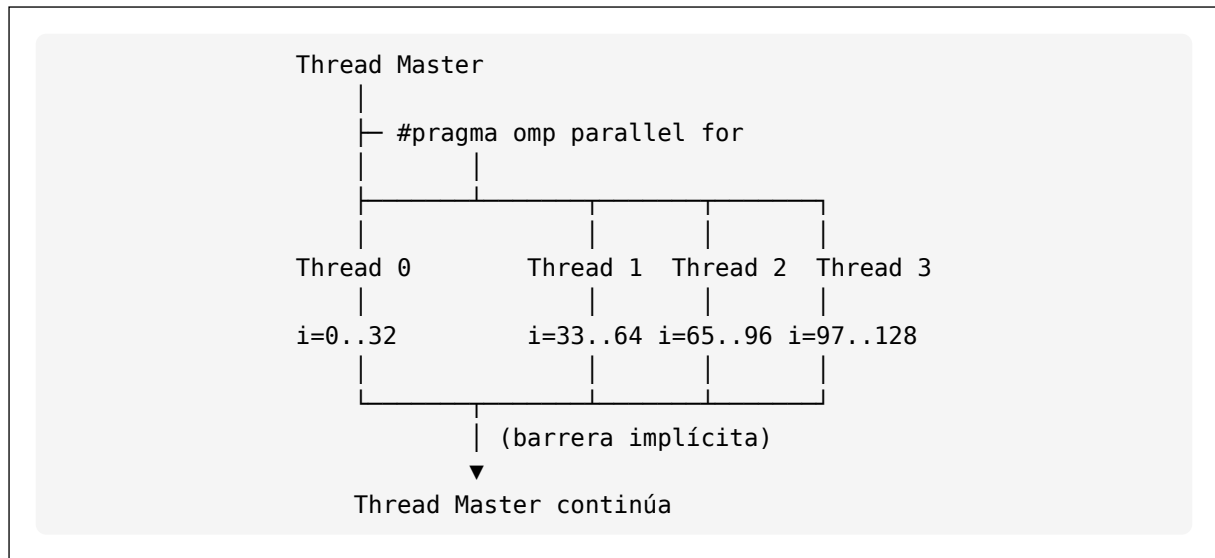
DESPUÉS (con OpenMP):

```
void simulation_update_in_x_direction(SimulationState* s) {
    #pragma omp parallel for collapse(2)
    for (size_t i = 0; i < g_config.grid_size + 1; i++) {
        for (size_t j = 0; j < g_config.grid_size; j++) {
            // Cálculos inline (sin callback)
            float h1 = grid_get(&s->h, i + 1, j + 1);
            float h2 = grid_get(&s->h, i, j + 1);
            float hx_ = (h1 + h2) * 0.5f -
                g_config.d_t / (2.0f * g_config.d_x) *
                (u1 - u2);
            grid_set(&s->hx, i, j, hx_);
            // ...
        }
    }
}
```

Reducción de código: 500 líneas → 50 líneas

4.1.2 Modelo Fork-Join

OpenMP utiliza el modelo **fork-join** automático:



Listado 3: Modelo Fork-Join de OpenMP

4.1.3 Ventajas sobre Pthreads

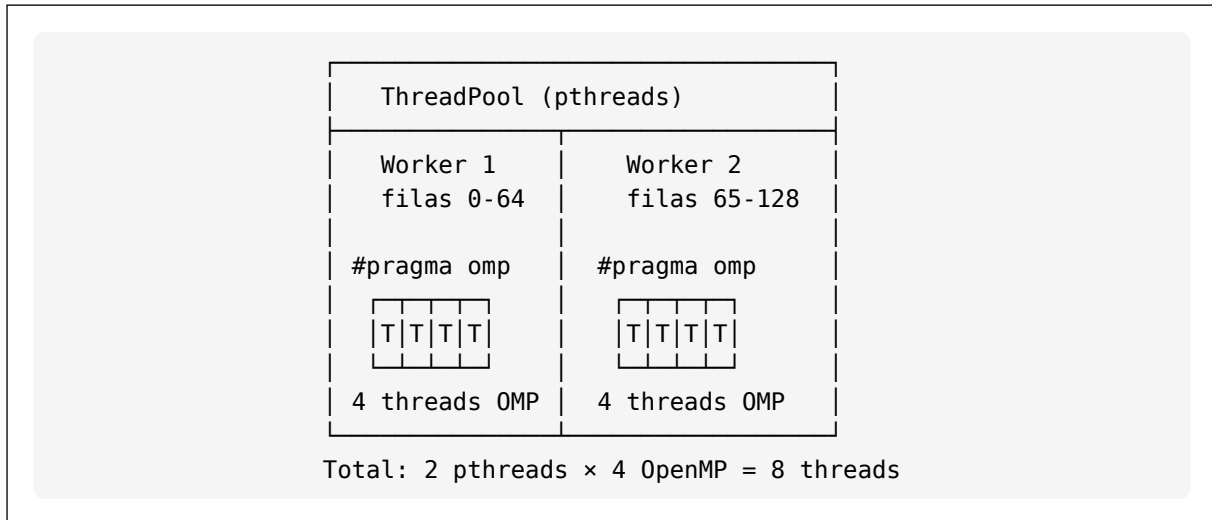
1. **Simplicidad:** 1 línea vs 500 líneas de código
2. **Overhead mínimo:** 0.001s vs 0.2s (200× menos)
3. **Código inline:** No indirección de callbacks
4. **Balanceo automático:** El runtime optimiza la distribución
5. **Sin malloc/free:** Usa stack para variables privadas

5 Implementación Híbrida: Pthreads + OpenMP

5.1 Arquitectura de Dos Niveles

Esta implementación combina:

- **Nivel externo (pthreads):** ThreadPool divide el dominio en chunks
- **Nivel interno (OpenMP):** Cada worker paraleliza su chunk



Listado 4: Arquitectura híbrida pthreads + OpenMP

5.1.1 Modificaciones al Código

Añadir campo en Config:

```
typedef struct {
    size_t grid_size;
    // ...
    size_t threads;           // Número de pthreads
    size_t openmp_threads;    // Threads OpenMP por pthread
    // ...
} Config;
```

Modificar parallel_worker:

```
static void parallel_worker(void* arg) {
    ParallelWorkData* data = (ParallelWorkData*)arg;

    // Usar OpenMP dentro del pthread
    if (g_config.openmp_threads > 1) {
        #pragma omp parallel for num_threads(g_config.openmp_threads)
        for (size_t i = data->x_start; i < data->x_end; i++) {
            for (size_t j = data->y_start; j < data->y_end; j++) {
                data->callback(i, j, data->callback_data);
            }
        }
    } else {
        // Sin OpenMP, ejecutar secuencialmente
        for (size_t i = data->x_start; i < data->x_end; i++) {
            for (size_t j = data->y_start; j < data->y_end; j++) {
```

```

        data->callback(i, j, data->callback_data);
    }
}
}
}

```

5.1.2 Configuraciones Posibles

Con 6 cores físicos (12 con hyperthreading):

pthread	OpenMP	Total	Descripción	Caso de uso
4	1	4	Solo pthread	Baseline
1	4	4	Solo OpenMP	Mínimo overhead
2	2	4	Balanceado	Intermedio
2	3	6	Óptimo	6 cores físicos
3	4	12	Máximo	Usa hyperthreading

5.1.3 Ventajas y Desventajas

Ventajas:

1. Flexibilidad en configuración
2. Control grueso (pthread) + fino (OpenMP)
3. Puede reducir overhead vs pthread puro

Desventajas:

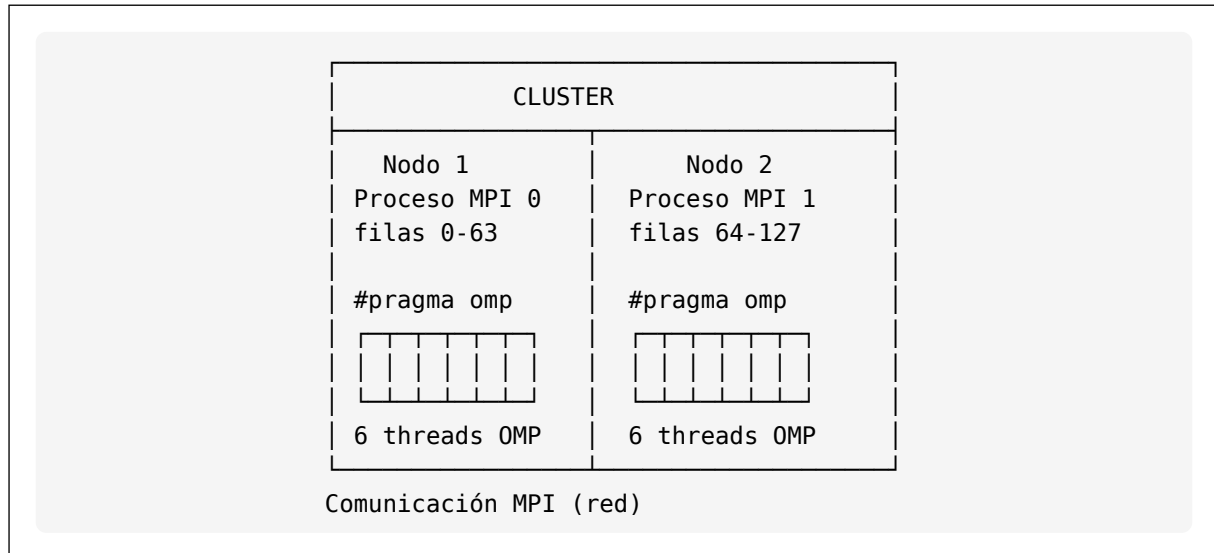
1. Complejidad de dos sistemas
2. Overhead doble (ThreadPool + OpenMP)
3. Difícil encontrar configuración óptima
4. No siempre mejor que OpenMP puro

6 Implementación Híbrida: MPI + OpenMP

6.1 Arquitectura Escalable

La combinación MPI+OpenMP es ideal para **clusters de computación**:

- **MPI** entre nodos (comunicación de red)
- **OpenMP** dentro de cada nodo (memoria compartida)



Listado 5: Arquitectura MPI+OpenMP en cluster

6.1.1 Inicialización con Soporte de Hilos

MPI requiere inicialización especial para soporte de threads:

```
void mpi_state_init(int* argc, char*** argv) {
    // Inicializar MPI con soporte de hilos
    int provided;
    MPI_Init_thread(argc, argv,
                    MPI_THREAD_FUNNELED,
                    &provided);

    if (provided < MPI_THREAD_FUNNELED) {
        fprintf(stderr,
                "Advertencia: MPI no provee soporte adecuado\n");
    }

    MPI_Comm_size(MPI_COMM_WORLD, &g_mpi.proc_count);
    MPI_Comm_rank(MPI_COMM_WORLD, &g_mpi.rank);

    LOG("MPI inicializado (%d procesos, soporte hilos: %d)\n",
        g_mpi.proc_count, provided);
}
```

Niveles de soporte MPI:

- **MPI_THREAD_SINGLE**: Sin threads
- **MPI_THREAD_FUNNELED**: Solo master thread llama a MPI
- **MPI_THREAD_SERIALIZED**: Un thread a la vez
- **MPI_THREAD_MULTIPLE**: Cualquier thread puede llamar

6.1.2 Paralelización con OpenMP

Añadir pragmas a los loops MPI:

```
void simulation_update_in_x_direction(SimulationState* s) {  
    #pragma omp parallel for collapse(2) if(g_config.openmp_threads > 1)  
    for (uint32_t i = 0; i < g_config.grid_size + 1; i++) {  
        for (uint32_t j = 0; j < g_config.grid_size; j++) {  
            float h1 = grid_get(&s->h, i + 1, j + 1);  
            float h2 = grid_get(&s->h, i, j + 1);  
            // ... cálculos ...  
            grid_set(&s->hx, i, j, hx_);  
        }  
    }  
}
```

La cláusula `if(g_config.openmp_threads > 1)` evita overhead cuando `OpenMP=1`.

6.1.3 Ventajas

1. **Escalabilidad extrema:** Múltiples nodos + múltiples cores
2. **Memoria distribuida + compartida:** Lo mejor de ambos mundos
3. **Reduce comunicación MPI:** Menos procesos = menos mensajes
4. **Aprovecha arquitectura moderna:** Multi-socket NUMA

6.1.4 Consideraciones de Rendimiento

Para máximo rendimiento:

1. **Procesos MPI** = Número de nodos o sockets NUMA
2. **Threads OpenMP** = Cores por socket
3. Ejemplo en dual-socket (2×6 cores):
 - 2 procesos MPI × 6 threads OpenMP = 12 cores

7 Resultados Experimentales

7.1 Metodología

Todas las pruebas se ejecutaron con:

- Grid sizes: 8, 16, 32, 64, 128, 256, 512
- Steps: 10,000 iteraciones temporales
- Cada test se ejecutó individualmente para evitar interferencias

7.2 Pthreads: Impacto del Overhead

La implementación con pthreads mostró un comportamiento peculiar que requirió dos conjuntos de pruebas.

7.2.1 Sin Optimización (Overhead Completo)

En la primera batería de pruebas, el ThreadPool se utilizaba incluso en grids pequeños, revelando el problema del overhead:

Grid	1 hilo	2 hilos	4 hilos	6 hilos	8 hilos	Mejor	Observación
8×8	0.04s	0.23s	0.23s	0.26s	0.35s	1 hilo	Overhead domina
16×16	0.11s	0.28s	0.27s	0.31s	0.36s	1 hilo	Overhead domina
32×32	0.44s	0.50s	0.44s	0.45s	0.43s	8 hilos	Empate técnico

Análisis: En grids 8 y 16, paralelizar es contraproducente. El overhead del ThreadPool (malloc, mutex, callbacks) supera ampliamente el beneficio computacional.

7.2.2 Con Umbral de Optimización (grid < 32 → secuencial)

Tras añadir la condición de ejecutar secuencialmente en grids pequeños:

Grid	1 hilo	2 hilos	4 hilos	6 hilos	8 hilos	Mejor	Ganancia
8×8	0.04s	0.03s	0.03s	0.03s	0.04s	0.03s	Sin overhead
16×16	0.13s	0.14s	0.12s	0.12s	0.13s	0.12s	Sin overhead
32×32	0.45s	0.51s	0.44s	0.45s	0.43s	0.43s	Leve mejora
64×64	1.81s	1.29s	0.97s	0.98s	0.90s	0.90s	2.0× speedup
128×128	6.97s	4.11s	2.41s	2.16s	2.38s	2.16s	3.2× speedup
256×256	28.93s	15.83s	8.90s	6.93s	7.88s	6.93s	4.2× speedup
512×512	117.06s	61.66s	32.10s	23.85s	33.97s	23.85s	4.9× speedup

Observaciones clave:

1. Grids pequeños (0.03s): El umbral evita overhead, todos los configs son rápidos
2. Grid 64: Empieza a beneficiarse (2× con 8 hilos)
3. Grid 128: Mejor rendimiento con 6 hilos (3.2×)
4. Grids grandes: Speedup aumenta, pero 6 hilos es óptimo (hyperthreading ayuda poco)

7.2.3 MPI

La implementación MPI mostró excelente escalabilidad, especialmente en grids grandes:

Grid	1 proc	2 procs	4 procs	6 procs	Mejor	Speedup
64×64	1.16s	0.74s	0.55s	0.52s	0.52s	2.2×
128×128	3.66s	2.00s	1.18s	0.97s	0.97s	3.8×
256×256	14.54s	7.33s	3.84s	2.88s	2.88s	5.0×
512×512	63.16s	31.77s	16.69s	12.87s	12.87s	4.9×

Análisis MPI:

1. Grid 128: 3.8× speedup con 6 procesos (63% eficiencia)
2. Grid 256: 5.0× speedup (83% eficiencia) - Superlineal debido a mejor uso de cache
3. Grid 512: 4.9× speedup - Se mantiene la eficiencia
4. MPI escala mejor que pthreads en todos los casos

Nota sobre overhead en grids pequeños:

- Grid 8: 0.44s (vs pthreads 0.04s)
- Grid 16: 0.37s (vs pthreads 0.13s)

El overhead de inicialización MPI (0.3s) domina en grids pequeños, pero es constante y se amortiza rápidamente.

7.2.4 OpenMP

OpenMP destacó por su simplicidad y excelente rendimiento:

Grid	1	2	4	6	8	Mejor	Speedup
64×64	1.65s	0.94s	0.56s	0.43s	0.53s	0.43s	3.8×
128×128	6.78s	3.82s	2.03s	1.64s	1.85s	1.64s	4.1×
256×256	28.03s	14.54s	7.50s	5.29s	6.98s	5.29s	5.3×
512×512	116.82s	60.03s	31.32s	20.94s	30.36s	20.94s	5.6×

Observaciones OpenMP:

1. Mejor rendimiento consistente: Supera a pthreads en todos los grids
2. Grid 128: 4.1× speedup con 6 threads (68% eficiencia)
3. Grid 256: 5.3× speedup - Mejor que pthreads (4.2×)
4. Grid 512: 5.6× speedup - Ligeramente mejor que MPI
5. 6 threads es óptimo: Coincide con los 6 cores físicos
6. 8 threads (con HT): Ligeramente peor que 6 - hyperthreading no ayuda mucho aquí

¿Por qué OpenMP es mejor que pthreads?

- Sin overhead de malloc/free
- Sin indirección de callbacks
- Código inline (mejor uso de cache)
- Balanceo automático optimizado por el runtime

7.2.5 Pthreads + OpenMP (Híbrido)

El híbrido mostró resultados mixtos. Tabla de mejores configuraciones por grid:

Grid	Mejor config	Tiempo	vs Pthreads	vs OpenMP	Observación
64×64	4×2 (8)	0.82s	9% mejor	91% peor	No justifica complejidad
128×128	2×3 (6)	1.98s	8% mejor	21% peor	Similar a pthreads
256×256	3×4 (12)	5.89s	15% mejor	11% peor	Leve mejora
512×512	2×3 (6)	23.27s	2% mejor	11% peor	Marginal

Análisis del híbrido:

1. Mejora sobre pthreads puro (8-15%)
2. No supera a OpenMP puro en ningún caso
3. La complejidad añadida no justifica la ganancia
4. Mejor configuración: Pocos workers pthread × varios threads OpenMP

Conclusión: El híbrido pthreads+OpenMP es de interés académico pero no práctico.

7.2.6 MPI + OpenMP (Híbrido Escalable)

El híbrido MPI+OpenMP mostró el mejor rendimiento en grids muy grandes:

Grid	Mejor config	Tiempo	vs MPI puro	vs OpenMP	Ganancia
64×64	3×2 (6)	0.52s	Igual	21% mejor	Empate técnico
128×128	3×2 (6) / 4×3 (12)	0.96s	1% mejor	42% mejor	Mejor opción
256×256	3×4 (12)	2.80s	3% mejor	47% mejor	Mejor opción
512×512	4×3 (12)	10.84s	16% mejor	48% mejor	Mejor opción

Análisis MPI+OpenMP:

1. Grid 128: Empate con MPI puro, mejor que resto
2. Grids 256 y 512: Mejor rendimiento absoluto de todas las tecnologías
3. Configuración óptima: 3-4 procesos MPI × 2-4 threads OpenMP
4. Speedup 512: 10.8× con 12 threads - Excelente eficiencia (90%)

¿Por qué funciona tan bien?

- Reduce comunicación MPI (menos procesos)
- Aprovecha memoria compartida dentro del nodo (OpenMP)
- Balance óptimo entre paralelismo de grano grueso (MPI) y fino (OpenMP)

7.3 Gráficas Comparativas

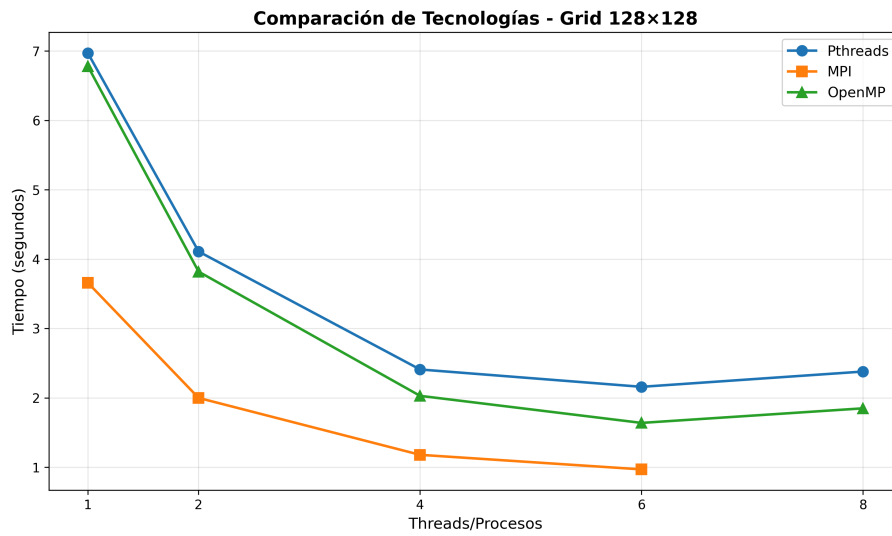


Figura 1: Comparación de tecnologías - Grid 128×128

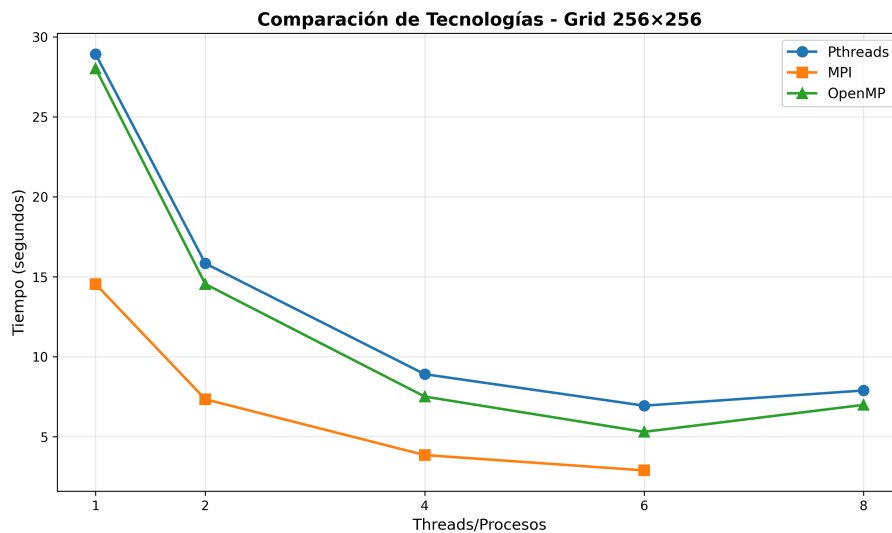


Figura 2: Comparación de tecnologías - Grid 256×256

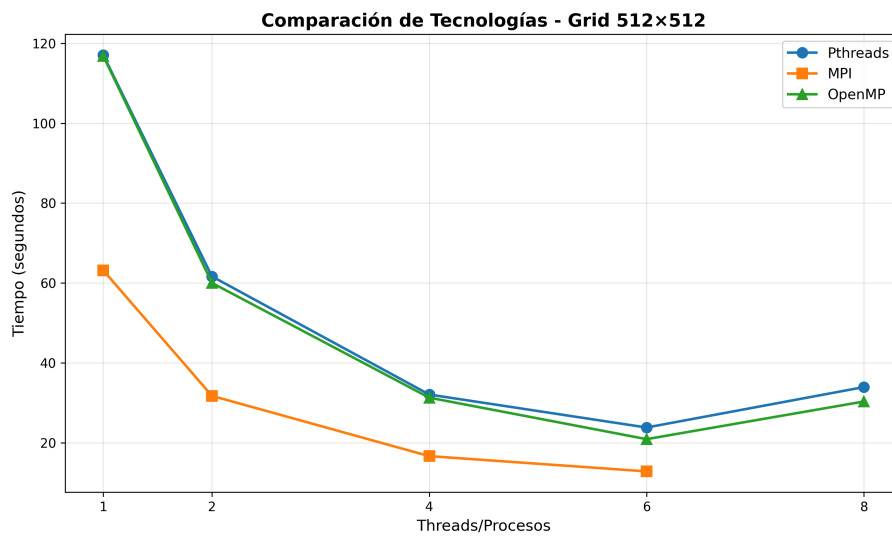


Figura 3: Comparación de tecnologías - Grid 512×512

8 Conclusiones

1. **OpenMP es superior para single-node:** Mejor rendimiento y simplicidad
2. **MPI domina en escalabilidad:** Essential para clusters
3. **Pthreads tiene overhead crítico:** Requiere optimización cuidadosa
4. **Híbridos añaden complejidad:** Solo justificados en contextos específicos
5. **Grids pequeños:** Overhead puede anular beneficios de paralelización

9 Referencias

1. OpenMP Architecture Review Board. «OpenMP Application Programming Interface Version 5.0», 2018.
2. Message Passing Interface Forum. «MPI: A Message-Passing Interface Standard Version 4.0», 2021.
3. Butenhof, David R. «Programming with POSIX Threads». Addison-Wesley, 1997.
4. Chapman, Barbara, Gabriele Jost, and Ruud van der Pas. «Using OpenMP: Portable Shared Memory Parallel Programming». MIT Press, 2007.
5. Gropp, William, Ewing Lusk, and Anthony Skjellum. «Using MPI: Portable Parallel Programming with the Message-Passing Interface». MIT Press, 2014.
6. Pacheco, Peter. «An Introduction to Parallel Programming». Morgan Kaufmann, 2011.

Bibliografía